

Page 1: Introduction to Python

Python is a high-level programming language designed with an emphasis on readability and developer productivity. Its syntax is relatively concise, which makes it useful for beginners as well as experienced developers. Python programs can be used for scripting, automation, web development, scientific computing, data analysis, and machine learning.

Additional Notes

- Key idea: Introduction to Python is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 2: Python Data Types

Python provides common data types such as integers, floating-point numbers, strings, booleans, lists, tuples, sets, and dictionaries. Lists are ordered mutable collections, tuples are ordered immutable collections, sets contain unique values, and dictionaries store key-value pairs. Choosing the appropriate data structure makes programs easier to understand and can improve performance.

Additional Notes

- Key idea: Python Data Types is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 3: Python Functions

A function is a reusable block of code that performs a particular operation. Functions can accept parameters and return values. Breaking a program into small functions improves readability, testing, maintenance, and reuse. Functions can also have default parameters and keyword arguments.

Additional Notes

- Key idea: Python Functions is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 4: Python Modules

A module is a Python file containing reusable definitions such as functions, classes, and variables. Modules help organize larger applications into logical components. Python also provides a large standard library, while third-party packages can be installed when additional functionality is needed.

Additional Notes

- Key idea: Python Modules is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 5: Machine Learning Overview

Machine learning is a branch of artificial intelligence in which algorithms learn patterns from data. Instead of writing every rule manually, a machine learning system learns a relationship between inputs and outputs from examples. Common learning settings include supervised learning, unsupervised learning, and reinforcement learning.

Additional Notes

- Key idea: Machine Learning Overview is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 6: Machine Learning Pipeline

A typical machine learning pipeline includes data collection, data cleaning, preprocessing, feature engineering, dataset splitting, model training, validation, evaluation, and deployment. Each stage can affect final model quality. Reproducible pipelines make it easier to compare experiments and diagnose failures.

Additional Notes

- Key idea: Machine Learning Pipeline is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 7: Data Collection

Data collection is the process of obtaining information required for an analytical or machine learning task. Data may come from databases, files, APIs, sensors, applications, or public datasets. Before using collected data, practitioners should examine its quality, completeness, consistency, and relevance to the target problem.

Additional Notes

- Key idea: Data Collection is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 8: Data Preprocessing

Data preprocessing converts raw data into a form suitable for analysis or model training. Typical operations include handling missing values, removing duplicates, correcting invalid records, normalizing numerical values, and encoding categorical variables. Good preprocessing reduces noise and prevents avoidable model errors.

Additional Notes

- Key idea: Data Preprocessing is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 9: Feature Engineering

Feature engineering creates useful input variables from raw data. A feature can be transformed, combined, extracted, or selected based on domain knowledge. Examples include extracting day of week from a timestamp, calculating ratios, or converting text into numerical representations. Strong features can improve model performance without changing the model architecture.

Additional Notes

- Key idea: Feature Engineering is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 10: Training and Validation

A dataset is often divided into training, validation, and test portions. The training set is used to learn model parameters. The validation set helps select configurations and tune hyperparameters. The test set is reserved for estimating performance on unseen data. Keeping these roles separate helps reduce misleading evaluation.

Additional Notes

- Key idea: Training and Validation is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 11: Model Evaluation

Model evaluation depends on the task. Classification commonly uses accuracy, precision, recall, F1 score, and confusion matrices. Regression can use mean absolute error, mean squared error, or related measures. Evaluation should reflect the actual business or application objective rather than relying on a single number.

Additional Notes

- Key idea: Model Evaluation is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 12: Embeddings

An embedding represents an object such as text as a numerical vector. Similar concepts can be represented by vectors that are close in an embedding space. Text embeddings are useful for semantic search, recommendation systems, clustering, retrieval-augmented generation, and classification.

Additional Notes

- Key idea: Embeddings is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 13: Vector Similarity

Vector search compares an input vector with stored vectors. Common similarity measures include cosine similarity, dot product, and Euclidean distance. Cosine similarity focuses on the angle between vectors, while dot product also depends on vector magnitude. When vectors are normalized, cosine similarity and dot product become equivalent.

Additional Notes

- Key idea: Vector Similarity is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 14: FAISS

FAISS is a library for efficient similarity search over dense vectors. A vector index stores numerical representations and can retrieve vectors close to a query vector. FAISS is commonly used as a retrieval component in applications that need fast nearest-neighbor search over collections of embeddings.

Additional Notes

- Key idea: FAISS is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 15: RAG Introduction

Retrieval-Augmented Generation, or RAG, combines information retrieval with language generation. Instead of asking a language model to answer only from its learned parameters, the application first retrieves relevant information from an external collection and provides that information as context to the model.

Additional Notes

- Key idea: RAG Introduction is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 16: RAG Ingestion

The ingestion stage prepares source documents for retrieval. Files are loaded, normalized, divided into chunks, converted into embeddings, and stored in an index. Additional metadata can be retained so that retrieved chunks can be traced back to their source documents.

Additional Notes

- Key idea: RAG Ingestion is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 17: Text Chunking

Chunking divides long documents into smaller pieces. Very large chunks can contain unrelated information and may exceed model context limits. Very small chunks can lose important context. Overlap between adjacent chunks can preserve information that crosses a chunk boundary. Recursive strategies can favor paragraph and sentence boundaries.

Additional Notes

- Key idea: Text Chunking is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 18: Dense Retrieval

Dense retrieval converts a query and documents into embedding vectors and searches for semantically similar vectors. It can retrieve passages that use different words but express related concepts. Dense retrieval is especially useful for conceptual questions and paraphrased queries.

Additional Notes

- Key idea: Dense Retrieval is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 19: BM25 Retrieval

BM25 is a lexical retrieval algorithm based on term frequency, inverse document frequency, and document-length normalization. It is strong when exact words, names, identifiers, or technical terms matter. Unlike dense retrieval, BM25 does not require semantic embeddings.

Additional Notes

- Key idea: BM25 Retrieval is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 20: Hybrid Retrieval

Hybrid retrieval combines dense and lexical search. Dense retrieval contributes semantic matching, while BM25 contributes exact term matching. A fusion method such as Reciprocal Rank Fusion can combine ranked results without requiring the raw scores from different retrieval systems to be on the same numerical scale.

Additional Notes

- Key idea: Hybrid Retrieval is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 21: Reciprocal Rank Fusion

Reciprocal Rank Fusion combines multiple ranked lists using the reciprocal of a document's rank. A common formulation adds 1 divided by k plus rank for each retrieval system in which the document appears. The approach is useful because ranking positions are comparable even when the underlying retrieval scores are not.

Additional Notes

- Key idea: Reciprocal Rank Fusion is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 22: Cross Encoder Reranking

A cross-encoder receives a query and candidate document together and produces a relevance score. Because both texts are processed jointly, the model can capture fine-grained interactions between query terms and document content. The trade-off is computational cost, so reranking is usually applied only to a small candidate set.

Additional Notes

- Key idea: Cross Encoder Reranking is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 23: Knowledge Graphs

A knowledge graph represents entities as nodes and relationships as edges. For example, a machine learning graph might contain nodes for Python, embeddings, FAISS, and RAG, connected by relationships describing how these concepts interact. Graph structure can support relationship and multi-hop questions.

Additional Notes

- Key idea: Knowledge Graphs is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 24: Graph Retrieval

Graph retrieval can identify entities mentioned or implied by a query, locate matching graph nodes, expand a neighborhood around those nodes, and rank useful relationships. A graph is particularly helpful when the question asks how multiple entities are connected rather than asking for one isolated fact.

Additional Notes

- Key idea: Graph Retrieval is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.

Page 25: RAG Evaluation

RAG systems should be evaluated at the retrieval and generation levels. Retrieval evaluation can include hit rate, precision, recall, mean reciprocal rank, and normalized discounted cumulative gain. Generation evaluation can examine relevance, faithfulness, completeness, and answer quality. A combination of automated metrics and qualitative review gives a more complete picture.

Additional Notes

- Key idea: RAG Evaluation is an important component of a practical information-processing system.
- Design consideration: The component should be simple enough to test independently and explicit enough that its inputs and outputs are easy to understand.
- Practical workflow: Define the input, perform the transformation, validate the result, and preserve useful metadata for debugging and evaluation.
- Common mistake: Treating one component as a complete solution can hide failures elsewhere in the pipeline. Retrieval, storage, ranking, and generation should be evaluated separately.